J3　情報システム工学実験実習Ｉ　実験報告書

題目　　統合開発環境

実施年月日　2024 年　7 月　18 日

2024 年　7 月　25 日

2024 年　8 月　7 日

提出年月日　2024 年　9 月　24 日

提出者

学籍番号　22139　氏名　福井　環

実験実習の概要

今回の実験では，IDE を使用する．1 週目では，ファイルの圧縮・展開の必要性を学ぶ．大量のデータを効率よく管理するには，圧縮や展開が必要になる．今回は Linax の主な圧縮方法の，xz，gz，bz2 を使用する．ファイルの圧縮後，展開し，元の状態に戻す．圧縮後の府いるサイズの違いや圧縮にかかった時間を調べる．2 週目，3 週目では IDE の基本的な使い方を学ぶ．主に，プロジェクトの作成プ，ラグインのインストール，プログラムの作成，デバッグ等を実践する．

1. 目的

- 1 週目　大量のファイルをやりとりするためにファイルの圧縮と展開を理解する.
- 2 週目　Linux のコマンドと bash（シェル）を組み合わせ，概要的な部分を理解し，引き出し（技術・知識）のひとつとして活用できること.
- 3 週目　Visual Studio Code を使って，IED の基本的な使い方を C 言語プログラムを通じて学ぶ.

2. 実験実習の内容

- 1 週目　コマンドを入力して演習する.
- 2 週目　Linux のコマンドと bash（シェル）組み合わせて，効果的に Linux の操作をするための方法，プログラムを行う.
- 3 週目　プロジェクトの作成，プラグインのインストール，プログラムの作成，デバック等を実践する.

3. 環境構築の手順

```
j22139@raspberrypi:~ $ mkdir -p ~/3j_jikken/IDE
j22139@raspberrypi:~ $ cd 3j_jikken/IDE/
j22139@raspberrypi:~/3j_jikken/IDE $ mkdir compression_and_decompression
j22139@raspberrypi:~/3j_jikken/IDE $ mkdir shell_scripts
j22139@raspberrypi:~/3j_jikken/IDE $ mkdir c_programming
j22139@raspberrypi:~/3j_jikken/IDE $ ls
c_programming  compression_and_decompression  shell_scripts
```

図 1　作業のためのディレクトリの作成

- 追加でインストールしたソフトウェア

  Visual Studio Code
- インストールコマンド

  $~ sudo apt -y install code

4. ファイルの圧縮と展開

4.1 課題 1

- 圧縮

表 1　圧縮時のサイズと時間と圧縮率

圧縮形式	ファイルサイズ(k)	時間(s)	圧縮率
元のファイル	184		
xz	64	0.271	35%
gzip	79	0.101	43%
bzip2	63	0.097	34%
zip	79	0.044	43%

```
j22139@raspberrypi:~/3j_jikken/IDE/compression_and_decompression $ time xz -k gakumonno_susume.txt

real    0m0.271s
user    0m0.179s
sys     0m0.084s
j22139@raspberrypi:~/3j_jikken/IDE/compression_and_decompression $ time gzip -k gakumonno_susume.txt

real    0m0.101s
user    0m0.085s
sys     0m0.013s
j22139@raspberrypi:~/3j_jikken/IDE/compression_and_decompression $ time bzip2 -k gakumonno_susume.txt

real    0m0.097s
user    0m0.083s
j22139@raspberrypi:~/3j_jikken/IDE/compression_and_decompression $ time zip -k gakumonno_susume.txt.zip gakumonno_susume.txt
  adding: GAKUMONN.TXT (deflated 57%)

real    0m0.044s
user    0m0.040s
sys     0m0.004s
```

図 2　圧縮時のコマンド

- 展開

表 2　展開時の時間

展開形式	時間(s)
xz	0.036
gzip	0.019
bzip2	0.066
zip	0.024

```
j22139@raspberrypi:~/3j_jikken/IDE/compression_and_decompression $ time xz -d gakumonno_susume.txt.xz

real    0m0.036s
user    0m0.014s
sys     0m0.020s
j22139@raspberrypi:~/3j_jikken/IDE/compression_and_decompression $ time gzip -d gakumonno_susume.txt.gz

real    0m0.019s
user    0m0.014s
sys     0m0.006s
j22139@raspberrypi:~/3j_jikken/IDE/compression_and_decompression $ time bzip2 -d gakumonno_susume.txt.bz2

real    0m0.066s
user    0m0.052s
sys     0m0.012s
j22139@raspberrypi:~/3j_jikken/IDE/compression_and_decompression $ time unzip gakumonno_susume.txt.zip
Archive:  gakumonno_susume.txt.zip
  inflating: GAKUMONN.TXT

real    0m0.024s
user    0m0.016s
sys     0m0.007s
```

図 3　展開時のコマンド

比較の条件が同じになるように図のように全ての形式でのコマンドを一緒にした.

4.2 課題 2

- 圧縮

表 3　tar コマンドでの圧縮時のサイズと時間と圧縮率

圧縮形式	ファイルサイズ(k)	時間(s)	圧縮率
元のファイル	220		
xz	75	0.239	34%
gzip	90	0.104	41%
bzip2	73	0.078	33%
zip	90	0.092	41%

```
j22139@raspberrypi:~/3j_jikken/IDE/compression_and_decompression $ time xz -k aozorabunko_text_files.tar

real    0m0.239s
user    0m0.179s
sys     0m0.060s
j22139@raspberrypi:~/3j_jikken/IDE/compression_and_decompression $ time gzip -k aozorabunko_text_files.tar

real    0m0.104s
user    0m0.087s
sys     0m0.017s
j22139@raspberrypi:~/3j_jikken/IDE/compression_and_decompression $ time bzip2 -k aozorabunko_text_files.tar

real    0m0.078s
user    0m0.066s
sys     0m0.012s
j22139@raspberrypi:~/3j_jikken/IDE/compression_and_decompression $ time zip -k aozorabunko_text_files.tar.zip aozorabunko_text_files.tar
  adding: AOZORABU.TAR (deflated 59%)

real    0m0.092s
user    0m0.077s
sys     0m0.014s
```

図 4　tar コマンドを使った圧縮時のコマンド

- 展開

表 4　展開時の時間

展開形式	時間(s)
xz	0.048
gzip	0.023
bzip2	0.078
zip	0.03

```
j22139@raspberrypi:~/3j_jikken/IDE/compression_and_decompression $ time xz -dk aozorabunko_text_files.tar.xz

real    0m0.048s
user    0m0.036s
sys     0m0.010s
j22139@raspberrypi:~/3j_jikken/IDE/compression_and_decompression $ time gzip -dk aozorabunko_text_files.tar.gz

real    0m0.023s
user    0m0.017s
sys     0m0.006s
j22139@raspberrypi:~/3j_jikken/IDE/compression_and_decompression $ time bzip2 -dk aozorabunko_text_files.tar.bz2

real    0m0.078s
user    0m0.060s
sys     0m0.018s
j22139@raspberrypi:~/3j_jikken/IDE/compression_and_decompression $ time unzip -K aozorabunko_text_files.tar.zip
Archive:  aozorabunko_text_files.tar.zip
  inflating: AOZORABU.TAR

real    0m0.030s
user    0m0.012s
sys     0m0.017s
```

図 5　展開時のコマンド

比較の条件が同じになるように図のように全ての形式でのコマンドを一緒にした.

4.3 課題 3

表 5　展開圧縮時の時間

展開時の時間	圧縮時の時間
56.211s	24m42.412s

```
j22139@raspberrypi:~/3j_jikken/IDE $ time xz -d 2023-02-21-raspios-bullseye-armhf-lite.img.xz

real    0m56.211s
user    0m48.499s
sys     0m7.234s
```

図 6　展開時のコマンド

```
● maruboro@Air kadai3 % ls -lh 2023-02-21-raspios-bullseye-armhf-lite.img
  -rw-r--r--@ 1 maruboro_ staff   1.8G  9 24 11:29 2023-02-21-raspios-bullseye-armhf-lite.img
```

図 7　ファイルサイズの表示

```
j22139@raspberrypi:~/3j_jikken/IDE $ time xz -k 2023-02-21-raspios-bullseye-armhf-lite.img

real    24m42.412s
user    24m31.688s
sys     0m7.851s
```

図 8　圧縮時のコマンド

5.　シェルスクリプトのプログラミングの練習

5.1 課題 4

```
$ homesize.sh
1  #!/bin/bash
2
3  du -h ~ | tail -n 1
```

問題　出力　ターミナル　ポート

ターミナル

```
du: /Users/maruboro/Library/Containers/com.apple.stocks.widget/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.ScreenTimeWidgetApplication.ScreenTimeWidgetIntentsExtension/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.Notes.WidgetExtension/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.systempreferences.GeneralSettings/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.Safari.SafariWidgetExtension/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.LinkedNotesUIService/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.Preview/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.UsageTrackingAgent/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.Home.HomeWidget.Interactive/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.biome.BiomeStreams.BiomeLighthousePlugin/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.CoreRoutine.helperservice/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.siri.metrics.MetricsExtension/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.CryptoTokenKit.ctkcard.ctkcardtoken/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.stocks: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.lighthouse.IFTelemetrySELFIngestor/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.lighthouse.PnROnDeviceWorker/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.Safari.SafariLinkExtension/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.quicklook.ui.helper/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.Notes.IntentsExtension/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.ax.MauiTTSSupport.MauiAUSP/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.lemon.lvoverseas/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.weather.WeatherIntents/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.AirPlayUIAgent/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/C88B58B5-711E-416D-9920-16AB2A04C3E0/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.lighthouse.BiomeLibraryEventUploader/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.voicebankingd/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.systempreferences/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.parsec.SafariAssistantWorker/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.podcasts.widget/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.mediaanalysisd/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.OSDUIHelper/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.ScreenContinuity/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.CalendarWidget.IntentsExtension/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.MobileSMS.MessagesActionExtension/Data: Operation not permitted
du: /Users/maruboro/Library/Containers/com.apple.Family/Data: Operation not permitted
du: /Users/maruboro/Library/ContainerManager: Operation not permitted
du: /Users/maruboro/Library/PersonalizationPortrait: Operation not permitted
du: /Users/maruboro/Library/Photos/Libraries/Syndication.photoslibrary: Operation not permitted
du: /Users/maruboro/Library/Metadata/CoreSpotlight: Operation not permitted
du: /Users/maruboro/Library/Metadata/com.apple.IntelligentSuggestions: Operation not permitted
du: /Users/maruboro/Library/Reminders: Operation not permitted
du: /Users/maruboro/Library/Cookies: Operation not permitted
du: /Users/maruboro/Library/CoreFollowUp: Operation not permitted
du: /Users/maruboro/Library/StatusKit: Operation not permitted
du: /Users/maruboro/Library/DoNotDisturb: Operation not permitted
du: /Users/maruboro/Library/Caches/com.apple.HomeKit: Operation not permitted
du: /Users/maruboro/Library/Caches/CloudKit: Operation not permitted
du: /Users/maruboro/Library/Caches/com.apple.Safari: Operation not permitted
du: /Users/maruboro/Library/Caches/com.apple.findmy.fmfcore: Operation not permitted
du: /Users/maruboro/Library/Caches/com.apple.containermanagerd: Operation not permitted
du: /Users/maruboro/Library/Caches/FamilyCircle: Operation not permitted
du: /Users/maruboro/Library/Caches/com.apple.homed: Operation not permitted
du: /Users/maruboro/Library/Caches/com.apple.findmy.fmipcore: Operation not permitted
du: /Users/maruboro/Library/Caches/com.apple.ap.adprivacyd: Operation not permitted
du: /Users/maruboro/.Trash: Operation not permitted
du: /Users/maruboro/Documents: Operation not permitted
 26G    /Users/maruboro
```

図 9　homesize

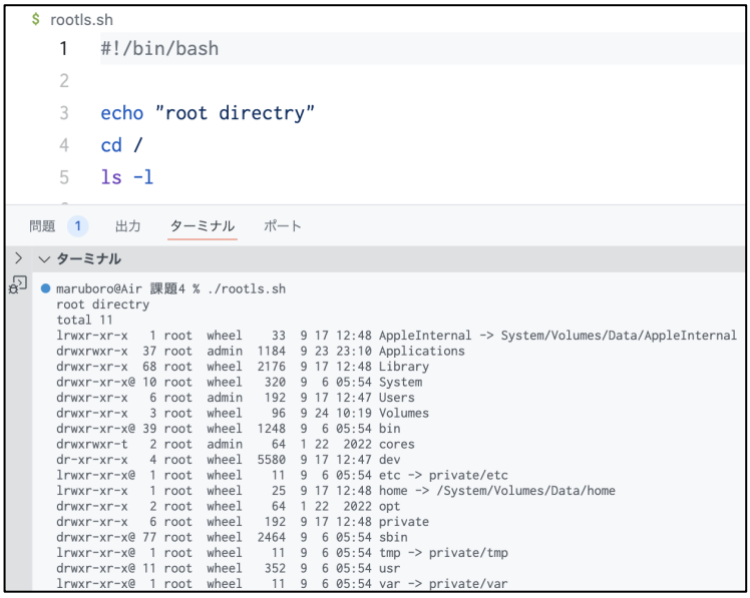

図 10　rootls

図 11　var

図 12　quote

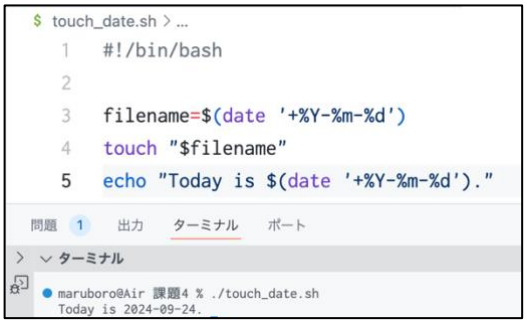

図 13　touch_date

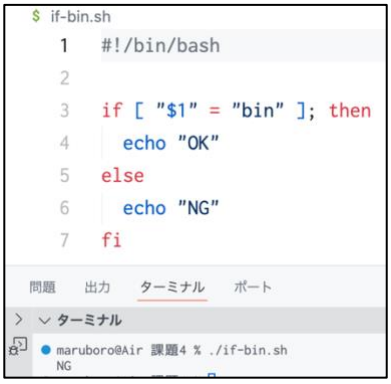

図 14　if-bin

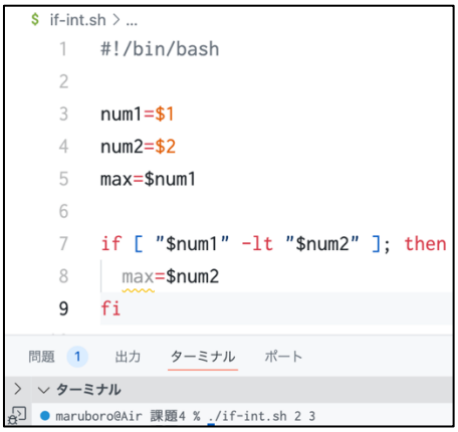

図 15　if-int

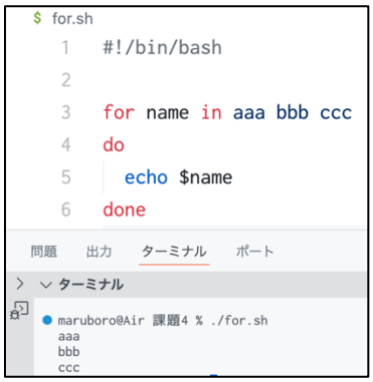

図 16　for

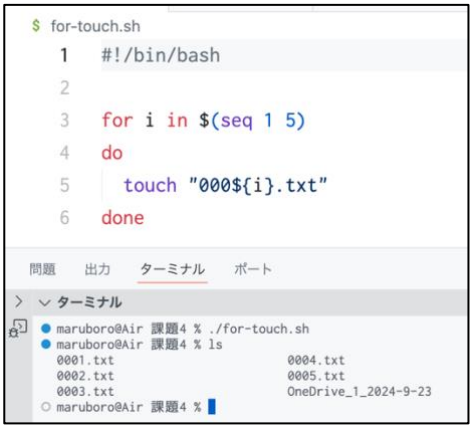

図 17　for-touch

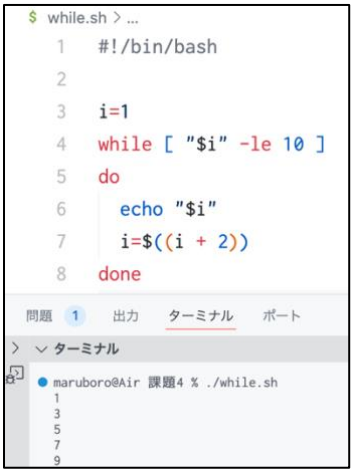

図 18　while

5.2 課題 5

classify_picture_sample.sh > target_dir

```
#!/bin/bash
#作業用ディレクトリの指定
target_dir="./test_text_files_1"
#findをかけるディレクトリの指定
#検索結果を絞り込む
#さらにパイプで読み込む
find "${target_dir}" -mindepth 1 -maxdepth 1 -name *.txt | while read path;
do
    echo "${path}"

    file=$(basename "${path}") #ファイル名の抽出

    year=${file:0:4}
    month=${file:4:2}
    day=${file:6:2}

    dir="${year}/${year}-${month}-${day}" #パスの作成

    echo "mkdir -p \"${dir}\""
    mkdir -p "${dir}" #ディレクトリの作成

    echo "cp \"${file}\" \"${dir}\""
    cp "${path}" "${dir}/" #ファイルのコピーまたは移動
done
```

図 19　作成したプログラム

```
ターミナル
maruboro@Air 課題5 % ./classify_picture_sample.sh
./test_text_files_1/20230312155103.txt
mkdir -p "2023/2023-03-12"
cp "20230312155103.txt" "2023/2023-03-12"
./test_text_files_1/20230312155115.txt
mkdir -p "2023/2023-03-12"
cp "20230312155115.txt" "2023/2023-03-12"
./test_text_files_1/20230312155101.txt
mkdir -p "2023/2023-03-12"
cp "20230312155101.txt" "2023/2023-03-12"
./test_text_files_1/20230312155247.txt
mkdir -p "2023/2023-03-12"
cp "20230312155247.txt" "2023/2023-03-12"
./test_text_files_1/20230312155053.txt
mkdir -p "2023/2023-03-12"
cp "20230312155053.txt" "2023/2023-03-12"
./test_text_files_1/20230312152507.txt
mkdir -p "2023/2023-03-12"
cp "20230312152507.txt" "2023/2023-03-12"
./test_text_files_1/20230312152505.txt
mkdir -p "2023/2023-03-12"
cp "20230312152505.txt" "2023/2023-03-12"
./test_text_files_1/20230312155243.txt
mkdir -p "2023/2023-03-12"
cp "20230312155243.txt" "2023/2023-03-12"
./test_text_files_1/20230312152510.txt
mkdir -p "2023/2023-03-12"
cp "20230312152510.txt" "2023/2023-03-12"
./test_text_files_1/20230312155055.txt
mkdir -p "2023/2023-03-12"
cp "20230312155055.txt" "2023/2023-03-12"
./test_text_files_1/20230312152748.txt
mkdir -p "2023/2023-03-12"
cp "20230312152748.txt" "2023/2023-03-12"
```

図 20　実行結果（test_text_files_1）

```
ターミナル
maruboro@Air 課題5 % ./classify_picture_sample.sh
./test_text_files_2/20230301121325.txt
mkdir -p "2023/2023-03-01"
cp "20230301121325.txt" "2023/2023-03-01"
./test_text_files_2/20230301053201.txt
mkdir -p "2023/2023-03-01"
cp "20230301053201.txt" "2023/2023-03-01"
./test_text_files_2/20230303174748.txt
mkdir -p "2023/2023-03-03"
cp "20230303174748.txt" "2023/2023-03-03"
./test_text_files_2/20230302161224.txt
mkdir -p "2023/2023-03-02"
cp "20230302161224.txt" "2023/2023-03-02"
./test_text_files_2/20230301221111.txt
mkdir -p "2023/2023-03-01"
cp "20230301221111.txt" "2023/2023-03-01"
./test_text_files_2/20230303060558.txt
mkdir -p "2023/2023-03-03"
cp "20230303060558.txt" "2023/2023-03-03"
./test_text_files_2/20230301214038.txt
mkdir -p "2023/2023-03-01"
cp "20230301214038.txt" "2023/2023-03-01"
./test_text_files_2/20230303040654.txt
mkdir -p "2023/2023-03-03"
cp "20230303040654.txt" "2023/2023-03-03"
./test_text_files_2/20230302072805.txt
mkdir -p "2023/2023-03-02"
cp "20230302072805.txt" "2023/2023-03-02"
./test_text_files_2/20230303211601.txt
mkdir -p "2023/2023-03-03"
cp "20230303211601.txt" "2023/2023-03-03"
```

図 21　実行結果（test_text_files_2）

```
ターミナル
mkdir -p "2023/2023-11-15"
cp "20231115221836.txt" "2023/2023-11-15"
./test_text_files_3/20230212050134.txt
mkdir -p "2023/2023-02-12"
cp "20230212050134.txt" "2023/2023-02-12"
./test_text_files_3/20231124001306.txt
mkdir -p "2023/2023-11-24"
cp "20231124001306.txt" "2023/2023-11-24"
./test_text_files_3/20230824143904.txt
mkdir -p "2023/2023-08-24"
cp "20230824143904.txt" "2023/2023-08-24"
./test_text_files_3/20230916200859.txt
mkdir -p "2023/2023-09-16"
cp "20230916200859.txt" "2023/2023-09-16"
./test_text_files_3/20230506185659.txt
mkdir -p "2023/2023-05-06"
cp "20230506185659.txt" "2023/2023-05-06"
./test_text_files_3/20231201085540.txt
mkdir -p "2023/2023-12-01"
cp "20231201085540.txt" "2023/2023-12-01"
./test_text_files_3/20230413130158.txt
mkdir -p "2023/2023-04-13"
cp "20230413130158.txt" "2023/2023-04-13"
./test_text_files_3/20231111215902.txt
mkdir -p "2023/2023-11-11"
cp "20231111215902.txt" "2023/2023-11-11"
./test_text_files_3/20230708072628.txt
mkdir -p "2023/2023-07-08"
cp "20230708072628.txt" "2023/2023-07-08"
./test_text_files_3/20231121212520.txt
mkdir -p "2023/2023-11-21"
cp "20231121212520.txt" "2023/2023-11-21"
./test_text_files_3/20230905111624.txt
mkdir -p "2023/2023-09-05"
cp "20230905111624.txt" "2023/2023-09-05"
./test_text_files_3/20231112080003.txt
mkdir -p "2023/2023-11-12"
cp "20231112080003.txt" "2023/2023-11-12"
./test_text_files_3/20230903172757.txt
mkdir -p "2023/2023-09-03"
cp "20230903172757.txt" "2023/2023-09-03"
./test_text_files_3/20231022044828.txt
mkdir -p "2023/2023-10-22"
cp "20231022044828.txt" "2023/2023-10-22"
./test_text_files_3/20230710061436.txt
mkdir -p "2023/2023-07-10"
cp "20230710061436.txt" "2023/2023-07-10"
./test_text_files_3/20230305173012.txt
mkdir -p "2023/2023-03-05"
cp "20230305173012.txt" "2023/2023-03-05"
./test_text_files_3/20230629155429.txt
mkdir -p "2023/2023-06-29"
cp "20230629155429.txt" "2023/2023-06-29"
./test_text_files_3/20230819132458.txt
mkdir -p "2023/2023-08-19"
cp "20230819132458.txt" "2023/2023-08-19"
./test_text_files_3/20230915012023.txt
mkdir -p "2023/2023-09-15"
cp "20230915012023.txt" "2023/2023-09-15"
./test_text_files_3/20230308234309.txt
mkdir -p "2023/2023-03-08"
cp "20230308234309.txt" "2023/2023-03-08"
./test_text_files_3/20231118111755.txt
mkdir -p "2023/2023-11-18"
cp "20231118111755.txt" "2023/2023-11-18"
```

図 22　実行結果（test_text_files_3）

```
j22139@raspberrypi:~/3J_jikken/IDE/shell_scripts/kadai-5 $ tree test_text_files_3 | head -20
test_text_files_3
├── 20230101002114.txt
├── 20230101011255.docx
├── 20230101011948.txt
├── 20230101014908.txt
├── 20230101031916.txt
├── 20230101032524.txt
├── 20230101042546.txt
├── 20230101044106.txt
├── 20230101055433.txt
├── 20230101060708.txt
├── 20230101065535.txt
├── 20230101065824.txt
├── 20230101065928.txt
├── 20230101071516.docx
├── 20230101084538.txt
├── 20230101090243.txt
├── 20230101095312.jpg
├── 20230101102841.txt
├── 20230101110205.txt
```

図 23　test_text_files_3 の tree コマンド先頭 20 行

6.　C プログラミングの練習

インストールしたプラグイン

- C/C++
- Japanese Language Pack for Visual Studio Code
- Bash IDE

これらのプラグインをインストールしたことで，VS Code は以下のような機能が強化された．

1.　C/C++の環境開発が整い，効率的なコーディングやデバックが可能になった．
2.　日本語化により，UI が日本語で表示され VS Code が使いやすくなった．
3.　Bash スクリプトの開発が効率化し，エラーチェックやコード補完などが便利になった．

6.1 課題 6

```
C test_1.c > ...
1  #include <stdio.h>
2
3  int main(void)
4  {
5      printf("Hello, World!\n");
6
7      return 0;
8  }
問題　出力　ターミナル　ポート
ターミナル
maruboro@Air 課題6 % gcc test_1.c -o 1
maruboro@Air 課題6 % ./1
Hello, World!
```

図 24　test1

このプログラムは，C 言語で「Hello, World!」という文字列を画面に表示する単純なプログラムである．このプログラムを実行するには，まずテキストエディタを使ってソースコードを test_1.c という名前で保存する．次に，コンパイル作業を行う．コンパイルには gcc を使用する．ターミナルで gcc test_1.c -o 1 というコマンドを入力すると，ソースコードがコンパイルされて 1 という名前の実行ファイルが生成される．次に，生成された実行ファイルを実行する．ターミナルで ./1 と入力することで，この実行ファイルを実行できる．すると，プログラムによって「Hello, World!」というメッセージが画面に表示され，実行が終了する．

図 25　test2

このプログラムは、ユーザーから整数の入力を受け取り，その値を画面に表示するものである．このプログラムを実行するには，まずテキストエディタを使ってソースコードを test_2.c という名前で保存する．次に，コンパイル作業を行う．コンパイルには gcc を使用する．ターミナルで gcc test_2.c -o 2 というコマンドを入力すると，ソースコードがコンパイルされて 2 という名前の実行ファイルが生成される．次に，生成された実行ファイルを実行する．ターミナルで ./2 と入力することで，この実行ファイルを実行できる．そしてプログラムが実行され，画面に「v?:」というプロンプトが表示される．これはユーザーに対して，値を入力するよう促すメッセージである．ここで，キーボードから 4 と入力し，Enter キーを押すと，プログラムは入力された値 4 を変数 v に格納し，次に「v＝4」というメッセージが表示される．

```c
#include <stdio.h>

#define N 10

int main(void)
{
    int v[N] = {0};

    for (int i = 0; i < N; i++) {
        printf("v[%d]?:", i);
        scanf("%d", &v[i]);
    }

    for (int i = 0; i < N; i++) {
        printf("v[%d] = %d\n", i, v[i]);
    }

    return 0;
}
```

```
maruboro@Air 課題6 % gcc test_3.c -o 3
maruboro@Air 課題6 % ./3
v[0]?:1
v[1]?:2
v[2]?:3
v[3]?:4
v[4]?:5
v[5]?:6
v[6]?:7
v[7]?:8
v[8]?:9
v[9]?:0
v[0] = 1
v[1] = 2
v[2] = 3
v[3] = 4
v[4] = 5
v[5] = 6
v[6] = 7
v[7] = 8
v[8] = 9
v[9] = 0
```

図 26　test3

このプログラムは，10 個の整数をユーザーから入力して，それを配列に格納し，その内容を表示するものである．このプログラムを実行する手順としては，まず test_3.c というファイル名でプログラムを保存し，ターミナルで gcc test_3.c -o 3 というコマンドを使ってコンパイルする．このコマンドは test_3.c というソースファイルをコンパイルして 3 という名前の実行ファイルを生成する．次に，./3 というコマンドを実行してプログラムを起動する．プログラムが起動すると，ユーザーは v[0]?: から v[9]?: まで，合計 10 回，整数の入力を求められる．たとえば，ユーザーが順に 1,2,3…8,9, 0 までの整数を入力した場合，各入力はそれぞれ配列 v の 0 番目から 9 番目までの要素に格納される．その後，プログラムは配列 v の内容を v[0] = 1 から v[9] = 0 という形式で表示し，正常に終了する．

6.2 課題 7

seiseki_search.c > ...

```c
#include<stdio.h>
int main(void)
{
  int n;  //学生の数
  scanf("%d", &n);  //学生の数を入力

  int seiseki[10000][6];  //学籍番号と5教科の点数を保持する配列
  for(int i = 0; i < n; i++) {  //学生ごとのデータを入力
    for(int j = 0; j < 6; j++) {  //各学年の学生番号と点数を入力
      scanf("%d", &seiseki[i][j]);
    }
  }

  int search = 0; //検索する学籍番号
  int exstence = 0; //学籍番号が存在するかをカウントする変数
  scanf("%d", &search); //検索する学籍番号を入力

//学籍番号を検索
  for(int i = 0; i < n; i++) {
    if(seiseki[i][0] == search) { //学籍番号が一致時いた場合
      exstence ++;  //存在をカウント
      for(int j = 0; j < 6; j++) {  //学籍番号と点数を出力
        printf("%d ", seiseki[i][j]);
      }
    }
  }
//学籍番号が存在しなかった場合の処理
  if(exstence == 0) {
    printf("none"); //存在しない場合は"none"と出力
  }
  printf("\n"); //改行

  return 0;
}
```

図 27　作成したプログラム

```
● maruboro@Air 課題7 % ./seiseki
5
10 10 20 30 40 50
20 60 70 80 90 100
30 50 40 30 20 10
40 90 80 70 60 50
50 10 9 8 7 6
20
20 60 70 80 90 100
```

図 28　実行結果（検索してデータがある場合）

```
● maruboro@Air 課題7 % ./seiseki
3
1 50 60 70 80 90
2 100 90 80 70 60
3 20 40 60 80 100
5
none
```

図 29　実行結果（検索してデータがない場合）

このプログラムは，複数の学生の学籍番号と 5 教科の点数を入力として受け取り，特定の学籍番号に該当する学生の情報を検索して表示するものである．まず，最初に学生の数 n を入力し，続いて n 人分の学籍番号と 5 教科の点数を配列に格納する．次に，ユーザーが検索したい学籍番号を受け取り，プログラムはすべての学生データの中からその学籍番号を探す．もし該当する学籍番号が見つかれば，学籍番号とその学生の 5 教科の点数を順番に表示する．該当する学籍番号が見つからなければ，「none」と表示して，存在しないことを示す．

このプログラムは，与えられた仕様を満たしている．指定された学籍番号に一致するデータが存在する場合，そのデータを正しく出力し，存在しない場合には「none」と表示して，条件通りの結果を出力している．したがって，プログラムは正しく動作していると判断できる．

このプログラムでは，学籍番号と各教科の点数を管理するために 2 次元配列を使用している．2 次元配列を用いることで，各学生のデータをひとまとめに扱うことができ，データの入力や検索が簡単になる．また，学籍番号は配列の最初の列に格納することで，検索の際に学籍番号を特定しやすくしている．たとえば，if(seiseki[i][0] == search）という条件文を使うだけで，学籍番号の一致を確認できるため，コードがシンプルになっている．検索結果が見つかったかどうかを確認するために，exstence という変数を導入している．この変数は，検索対象の学籍番号が見つかったかどうかを判定する機能を果たしている．もし exstence が 0 のままなら，検索対象が見つからなかったことを意味し，その場合「none」

と表示する．

7. 検討課題

7.1 課題 8

(1) ファイルの圧縮・展開

ランレングス符号の特徴は，連続する同じデータの列を，データの値とその繰り返し回数で表現する手法である[1]．例えば，画像やテキストなどで同じデータが連続する場合に有効である[2]．しかし，圧縮率はデータの内容に依存し，ランダムなデータでは逆にデータが大きくなる可能性もある[2]．アルゴリズムの基本的な考え方は，まず入力データを順に読み，連続する同じ値が現れる区間を見つける[2]．そして，各区間のデータの値と繰り返し回数を組みにして出力する．

ハフマン符号の特徴は，文字や記号の出現頻度に応じて，可変長のビット列を割り当てる手法である[3]．出現頻度の高い文字には短いビット列を，低い文字には長いビット列を割り当てることで，全体の平均ビット数を減らす[3]．データ全体のエントロピーをもとに圧縮するので，一般的に高圧縮率を持つ[3]．アルゴリズムの基本的な考え方は，まず各文字の出現頻度を数え，頻度の低いものから順に 2 つ選んで,「ハフマン木」を構築する．次に，木の左側には 0，右側には 1 を割り当て，葉の文字までの道筋で符号を決定する．そして，ハフマン木をもとに，各文字に対応するビット列を用いてデータをエンコードする[3]．

辞書式符号の特徴は，データの中に現れるパターンを辞書（ルックアップテーブル）として保存し，パターンを辞書のインデックス（参照）置き換える手法である．LZ77 や LZ78, LZW（GIF などで使用される）などのアルゴリズムが代表例である．繰り返し出現するパターンを効率的に圧縮でき，特に大きなテキストやバイナリデータに有効である．アルゴリズムの基本的な考え方は，まず入力データを逐次読み，既存の辞書（パターン表）にデータの部分列が含まれているか確認する．そして，辞書に含まれている場合はそのパターンを辞書のインデックスで置き換え，含まれていない場合は新しいエントリとして追加する．

(2) 考察

課題 1 の結果から xz 形式は最も高い圧縮率を達成しており，ファイルサイズが最も小さくなっている．LZMA アルゴリズムの優れた圧縮性能による結果と考えられる．bzip2 形式は，xz 形式とほぼ同等の高い圧縮率を示しており，Burrows-Wheeler 変換を利用した高圧縮アルゴリズムの特性が現れている．gzip 形式と zip 形式は，両者とも同じ圧縮率であり，同じ DEFLATE アルゴリズムを使用しているため，圧縮効率が同じになることが確認できる．xz 形式は，最も圧縮時間が長い．これは、LZMA アルゴリズムが高い圧縮率を実現するために，多くの計算資源を消費するためである．gzip 形式と bzip2 形式は，gzip の方がやや短いが，どちらも圧縮時間は中程度．gzip は速度重視，bzip2 は圧縮率重視のアルゴリズムであるため，このような結果が得られたと考えられる．zip 形式は，圧縮時間が最も短

い．複数のファイルをまとめる目的に適しており，スピードを重視する場合に有効である．高圧縮率の xz 形式や bzip2 形式は，圧縮率が高い代わりに，圧縮時間が長くなる傾向にある．このため，ファイルサイズの削減が求められる場合には有効であるが，リアルタイム処理には不向きである．一方，gzip 形式や zip 形式は，圧縮率よりも圧縮速度を優先している．このため，リアルタイム性や圧縮/展開のスピードが求められる用途に適している．展開時間の短さは，ファイルを頻繁に解凍する用途では役立つ．例えば，gzip 形式のファイルは展開が非常に速いため，ログファイルや一時的なデータの圧縮に向いている．bzip2 形式のファイルは展開に時間がかかるため，一度圧縮したファイルを頻繁に解凍しない，アーカイブ目的などに適している．

課題 2 の結果から，課題 1 では単一のテキストファイルを対象としたのに対し，課題 2 では複数のファイルを含むディレクトリ全体を圧縮対象としたため，圧縮アルゴリズムの結果に違いが見られた．特に，圧縮後のファイルサイズや圧縮時間において，課題 2 ではファイル間の冗長性をうまく活用できたため，bzip2 形式がより高い圧縮効率を示した．一方で，gzip 形式は課題 1 と同様に展開時間が短く，リアルタイム性を重視するシステムにおいて有効であると考えられる．これらの結果から，データの性質や構造によって圧縮アルゴリズムの選択が大きく影響を受けることがわかる．

xz 形式は，高い圧縮率を実現するために LZMA（Lempel-Ziv-Markov chain algorithm）を使用している圧縮アルゴリズムである．その特徴には以下のような点がある．まず，大きなファイルに対して非常に優れた圧縮率を持ち，ストレージの節約に役立つ．圧縮後のファイルサイズは小さくなるものの，展開に要する時間は比較的長くなる傾向がある．圧縮には一定の時間がかかるため，特に大きなファイルの場合，圧縮時間が長くなることがあります．課題 3 の結果から展開に 56.211 秒を要したという結果は，1.8GB という大きなファイルサイズを考慮すると妥当であると言える．xz 形式は圧縮率が高い一方で，展開には時間がかかる．このため，展開に要した時間がこのファイルサイズに対して相対的に短すぎることはなく，むしろ妥当な結果であると考えられる．圧縮に 24 分 42.412 秒かかったというのは，特に大きなイメージファイルを圧縮する際には一般的な時間範囲内である．高圧縮率を実現するために，圧縮処理は時間がかかることが多く，そのため，圧縮に要する時間が長くなるのは自然な結果である．今回の実験結果から，xz 形式の圧縮アルゴリズムは非常に高い圧縮率を実現する一方で，展開および圧縮に要する時間が比較的長いことがわかる．この特性は，ストレージ容量を重視する環境においては非常に有用だが，リアルタイムでの処理を求める場合には，他の圧縮アルゴリズム（例えば、gzip など）の方が適している可能性があると考えられる．

7.2 課題 9

シェルスクリプトを用いて，サーバーやサービスの起動，停止，再起動を自動化できる．これを利用した事例，Web サーバーの起動スクリプトを図に示す．

```
#!/bin/bash
# Apacheを起動するスクリプト

systemctl start apache2
echo "Apache server started."
```

図 30　Web サーバーの起動スクリプト

また，cron ジョブを使用して，シェルスクリプトを定期的に実行することができる．これにより，システムの状態を監視したり，ログファイルをチェックしたりできる．そのため，CPU 使用率の監視スクリプトに使用され，事例を図に示す．

```
#!/bin/bash
# CPU使用率を監視するスクリプト

CPU_USAGE=$(top -bn1 | grep "Cpu(s)" |
sed "s/.*, *\([0-9.]*\)%* id.*/\1/" |
awk '{print 100 - $1}')
if (( $(echo "$CPU_USAGE > 80" | bc
-l) )); then
  echo "High CPU usage detected:
$CPU_USAGE%"
fi
```

図 31　CPU 使用率の監視スクリプト

また，データの整理や定期的なバックアップを実行することができる．事例を図に示す．

```
#!/bin/bash
# データをバックアップするスクリプト

tar -czf /backup/mydata_$(date +
%F).tar.gz /path/to/data
echo "Backup created successfully."
```

図 32　データのバックアップスクリプト

また，定型的な処理を自動化することで，時間を節約できる．複数のファイルのリネームをするのに使用されている．

```
#!/bin/bash
# 拡張子を変更するスクリプト

for file in *.txt; do
  mv "$file" "${file%.txt}.bak"
done
echo "All .txt files have been renamed
to .bak"
```

図 33　複数のファイルのリネーム

シェルスクリプトは、日常的なタスクを自動化し，効率を向上させるための強力なツールである．特に，サーバー管理やデータ処理の分野で広く活用されている．

7.3 課題 10

IDE の機能：コード補完

コード補完は，統合開発環境（IDE）における非常に重要な機能の一つである．この機能

は，開発者がコードを入力する際に，候補となるコードスニペットや関数名，変数名などを提案するものである．これにより，開発者は必要なコードを迅速に入力でき，タイプミスや構文エラーを減少させることができる．

コード補完はソフトウェア開発プロセスにおいて，特にプログラミングの効率を大幅に向上させる役割を果たす．開発者が必要な情報を手早く得られるため，学習曲線が緩和され，新しい言語やフレームワークへの適応が容易になる．特に大規模なプロジェクトでは，複雑なコードベースを扱うため，コード補完によって生産性が向上する．

コード補完は，ソフトウェアの品質向上にも寄与する．正確な候補を提案することで，開発者が行うタイプミスや誤った構文の入力を減らすことができ，これによりバグの発生率が低下する．また，標準的なライブラリや API を簡単に参照できるため，保守性が向上する．開発者が一貫したスタイルでコードを書くことが促され，可読性の向上にもつながる．

このように，コード補完はソフトウェア開発プロセスを円滑にし，ソフトウェアの品質を向上させる重要な機能となっている．

8. まとめ

今回の実験は，3 週間を通じてファイル管理と IDE の基本操作を学ぶことを目的としている．1 週目では，Linux 環境におけるファイル圧縮と展開の必要性を理解し，主な圧縮形式である xz，gz，bz，zip を用いて実際に圧縮・展開を行い，圧縮後のファイルサイズや処理時間を調べた．2 週目には，Linux のコマンドと bash シェルを組み合わせ，効率的な操作方法を学び，基本的なプログラム作成も行なった．3 週目では，Visual Studio Code を利用し，プロジェクトの作成，プラグインのインストール，プログラムの作成，デバッグなどの実践を通じて IDE の活用法を学んだ．3 週の実験を通じて，ファイル管理やプログラミングの基礎技術を学んで，今後の実践に活かすための知識を深めることができた．

9. 感想

この 3 週間の実験実習を通じて，Linux の基礎から C 言語のプログラミングまで幅広く学ぶことができてよかった．1 週目はファイルの圧縮と展開について理解し，大量のデータを効率的に扱うスキルが身についた．具体的なコマンドを入力しながらの演習は，理論だけではなくて実践的にできてよかった．2 週目では Linux のコマンドと Bash シェルを組み合わせることで，より効率的な操作ができるようになった．シェルスクリプトの基本を学ぶことで，日常的な作業の自動化にも繋がり，今後の作業がスムーズに進むと思った．3 週目は Visual Studio Code を用いて C 言語のプログラミングを学び，IDE はとても便利だと思った．全体を通して，実践的なスキルを得ることができ，これを生かして今よりも効率的に作業を進めていきたいと思った．

参考文献

[1] 「【高校情報１】ハフマン法・ランレングス法（可逆圧縮法）と非可逆圧縮」, 高校情報科・情報処理技術者試験対策の突破口ドットコム, https://note.com/toppakou/n/nbf4128366f1e, 2024 年 9 月 24 日参照.

[2] 「ランレングス符号化の使い方とメリットを解説！」, IT の学び, https://itmanabi.com/run_length_encoding/, 2024 年 9 月 24 日参照.

[3] 矢沢　久雄, 「アルゴリズム問題　出題テーマの仕組みがわかれば 50%以上正解できる「ハフマン符号を用いた文字列圧縮」」, 独習ゼミ, https://www.seplus.jp/dokushuzemi/ec/fe/fenavi/algorithms_tips/huffman_code/#!, 2024 年 9 月 24 日参照.